DEVBRIDGE – VERIFIED-TALENT FREELANCE PLATFORM

Module 6

Complete Contract & Hypothesis Reference

Every rule the event-projection layer emits on, every skill and edge the ontology carries, every constant the scoring filter reads, every check the integrity layer runs, and every modelling hypothesis behind all of it – reproduced from the committed contract files, not summarised or paraphrased away. Where a value has a stated sweep range or a rejected alternative, both are reproduced here too.

Author	*[Module 6 owner — fill in name / roll number]*
Scope	Submodules 6.0–6.5: projection, ontology, ingestor, scoring, query, integrity
Contracts	`projection-rules.yaml` v1.0.0 · `m6-parameters.yaml` v1.0.0 `ontology-seed.yaml` v0.1.0 · `integrity-rules.yaml` v1.0.0
Hypotheses	Decision records O1–O8, `docs/docs/m6-decisions.html`
Date	September 14, 2026

Nothing in this document is fitted. Every constant below is a documented prior with a stated sweep range, never a value tuned to make one candidate look right.

Contents

1 Scope and How to Read This Document **2**

2 Part I — The Modelling Hypotheses (O1–O8) **2**
- 2.1 O1 — The Observation Model . . . 2
- 2.2 O2 — Bounded Scale versus Unbounded Gaussian . . . 3
- 2.3 O3 — Multi-Skill Events and Relevance . . . 3
- 2.4 O4 — Out-of-Order Arrival . . . 3
- 2.5 O5 — Propagation Timing and Depth . . . 4
- 2.6 O6 — The Unset Constants . . . 4
- 2.7 O7 — Edge Types and γ . . . 4
- 2.8 O8 — Where New Skills Come From . . . 5
- 2.9 What the Resolved Model Produces — the Illustrative Check . . . 5

3 Part II — The Ontology Layer (6.1) **6**
- 3.1 The Three Volatility Tiers . . . 6
- 3.2 Every Skill in the Seed Ontology — All 27 . . . 6
- 3.3 Every Relation in the Seed Ontology — All 12 . . . 7

4 Part III — The Event Projection Layer (6.0) **8**
- 4.1 Every Emit Rule — All 7 . . . 8
- 4.2 Every Reliability Factor — All 5 . . . 9
- 4.3 Every Suppression Rule — All 6 . . . 9
- 4.4 Every Invariant — All 6 . . . 10
- 4.5 Machine-Readable Thresholds . . . 10

5 Part IV — The Scoring Parameters (6.3) **10**
- 5.1 Space . . . 11
- 5.2 Prior . . . 11
- 5.3 Observation . . . 11
- 5.4 Decay . . . 11
- 5.5 Propagation . . . 11
- 5.6 Display . . . 11

6 Part V — The Integrity Layer (6.5) **12**
- 6.1 Every Check — All 9 . . . 12
- 6.2 Reporting Policy . . . 13

7 Part VI — Reference Distributions **13**

8 Part VII — Validation of the Shipped Implementation **13**

9 Appendix — Every Constant, One Table **14**

1 Scope and How to Read This Document

This is a reference, not an explainer: it exists to enumerate *every* rule the skill-graph layer runs on, in one place, so a reviewer can check any single number against its source without reading YAML. Nothing here is invented for exposition – every table, threshold and rationale is reproduced from a committed contract file, named at the point it is used.

Source file	What it fixes
`docs/docs/m6-decisions.html`	The eight modelling hypotheses (O1–O8): what was undecided, what was chosen, and what was rejected and why.
`contracts/ontology-seed.yaml`	Every skill, its volatility tier, and every directed relation between skills.
`contracts/projection-rules.yaml`	How a verified artefact becomes a scored event: seven emit rules, five reliability factors, six suppressions, six invariants, and every numeric threshold used to classify evidence.
`contracts/m6-parameters.yaml`	Every free constant in the scoring filter, with a sweep range and a reason for each.
`contracts/integrity-rules.yaml`	The nine checks that describe the *shape* of a developer's evidence without ever touching a score.
`contracts/reference-distributions.yaml`	The empirical distribution the platform-rating observation is measured against.

Three refusals that shape every rule below

No model here is trained – every constant is declared with a reason. There is no overall score – every rule below produces evidence about ONE skill, never a verdict on a person. And the LLM that reads a CV upstream never reaches this layer – everything from here on is deterministic arithmetic over declared values.

2 Part I — The Modelling Hypotheses (O1–O8)

Eight questions the plan left open, each resolved to one concrete value with its reasoning and, where one existed, the alternative that was rejected and why. Every constant named here lives in `contracts/m6-parameters.yaml` or `contracts/ontology-seed.yaml`; this section is the reasoning behind those files, not a restatement of their values.

2.1 O1 — The Observation Model

DECIDED —

$\sigma^2_{\text{obs}} = S_0/(\text{reliability} \times \text{relevance} \times \text{strength})$, with $S_0 = 0.10$. Observed values come from percentiles against a reference distribution where one exists, and from documented ordinal bands where none does.

Two things were missing: how an event becomes a number the filter can absorb, and how confident the filter should be in it. Everything in the denominator of σ^2_{obs} arrives on the event itself, so S_0 is the only free constant – which keeps the sensitivity sweep small. High reliability, high relevance and high strength together mean low variance, and therefore a large update.

Event	ρ	rel.	str.	σ^2_{obs}	Weight against a fresh prior
`platform_rating` / algorithms	0.90	0.95	0.80	0.15	0.92 — dominates immediately, as an authoritative API should
`claim_corroborated` / typescript	0.48	1.00	0.70	0.30	0.85
`repo_artifact` / typescript	0.42	0.90	0.60	0.44	0.79
`artifact_live` / nextjs	0.51	0.60	0.40	0.82	0.67
`cv_claim` / numpy	0.20	1.00	0.30	1.67	0.50 — a bare claim moves the estimate least, by construction

The one real modelling choice is the **reference population for platform ratings**. Against the rated Codeforces cohort, a 1495 (Specialist) sits near the 60th percentile. But the population that matters for "algorithmic

ability" is developers generally, not the self-selected few thousand who compete – and being rated at all already clears the general-population floor. So observed $= 0.50 + 0.40 \times \text{ECDF}_{\text{rated}}(\text{rating})$, putting 1495 at 0.74. The ECDF is built once from the public user list and stored as a lookup table: a for-loop, not a training run (Section 7).

REJECTED ALTERNATIVE

Letting the model read the raw rating. Feeding 1495 directly, or min-max scaling it, makes the score depend on the numeric conventions of whichever platform reported it – a LeetCode contest rating and a Codeforces rating are not on the same scale and never will be. Percentile against a named reference population is the only mapping that survives adding a second platform.

2.2 O2 — Bounded Scale versus Unbounded Gaussian

DECIDED —

Run the filter in logit space. Display through the sigmoid. Report `sigma` as the display-space band half-width and `sigma_logit` as the state.

Ability displays in $[0, 1]$; a Kalman update on a bounded quantity can walk outside it. Filtering on $x = \text{logit}(\theta)$ makes the state genuinely unbounded, so the update is exact rather than approximately-valid-then-clipped. The lower confidence bound survives the transform intact because sigmoid is monotone: score $= \text{sigmoid}(\mu_x - z\sigma_x)$ is a real quantile, not an approximation of one.

DECIDED — This resolves an arithmetic mismatch that did not reconcile

The plan flagged that the worked `/explain` response reports `sigma 0.12` alongside `lambda 0.00021` over 587 days, when the decay term alone adds 0.123 to the variance. In logit space the numbers agree: take $\sigma_x(\tau) \approx 0.40$ before the silence; decay carries it to $\sqrt{0.16 + 0.123} = 0.53$; at $\theta = 0.68$ the display band is $\text{sigmoid}(0.7538) - \text{sigmoid}(0.7538 - 0.53) = 0.68 - 0.555 = \mathbf{0.125}$, against the published 0.12 and 0.56. That the reconciliation falls out of the logit choice rather than being fitted to it is the strongest evidence available that the choice is right.

REJECTED ALTERNATIVE

Clip to $[0, 1]$**.** Clipping biases every estimate near the ends toward the middle and silently destroys the variance – the exact quantity the whole design rests on. It would also make the decay story false at the top of the range, where a wide band is precisely what you want to see.

2.3 O3 — Multi-Skill Events and Relevance

DECIDED —

Relevance enters the observation variance, never the observed value. One event yields one observed value shared across its skills, with a different variance per skill.

Relevance is epistemic. A repository with `typescript:0.9, mongodb:0.6` is not saying "this demonstrates less MongoDB ability than TypeScript ability." It is saying *we are less sure this observation is about MongoDB at all* – uncertainty about what was observed, which belongs in the variance. Scaling the observed value instead would encode the wrong claim and would drag every incidental skill toward zero rather than leaving it uninformed. Because relevance already sits in the denominator from O1, this needs no new mechanism: a relevance-0.6 skill receives an update with $1.5\times$ the variance of a relevance-0.9 skill from the same event, and moves correspondingly less.

2.4 O4 — Out-of-Order Arrival

DECIDED —

Recompute from the log, ordered by $(\text{occurred_at}, \text{event_id})$. Stored state is a derived cache that may be deleted at any time without loss.

A 2024 repository can be observed in 2026, and sequential Kalman updates are order-dependent. Rather than patch the ordering problem, the decision removes it: ingest appends the event and marks the affected (developer, skill) pair dirty; a read folds the sorted log. Ties break on `event_id` so the fold is deterministic and two machines agree. The cost is $O(n)$ per skill per read, where n is tens of events at this scale – microseconds.

DECIDED — What this buys, and why it is the highest-leverage decision here

Ablation becomes a filter over the log rather than a re-collection. The parameter sweep becomes a loop over parameter sets against a fixed log. `as_of` queries become free. All three validation experiments cost days instead of weeks because of this one choice.

2.5 O5 — Propagation Timing and Depth

DECIDED —

Read time. One hop. Sourced only from direct-evidence states. Damping is applied as variance inflation, so propagation reuses the Kalman update rather than adding a second mechanism.

A neighbour's state enters as a pseudo-observation with variance $\sigma^2(\text{neighbour})/\gamma$ – a weaker edge means a noisier observation, which is what damping actually means. Write-time propagation would turn the graph into a cache needing invalidation on every arrival, and it interacts badly with O4: the system would be recomputing from the log *and* maintaining a derived overlay. Read time keeps one source of truth. **Sourcing only from direct-evidence states** is what makes this safe – if propagated states could themselves propagate, the computation would need iteration to a fixed point and could feed back around a cycle. Restricting sources to states with real evidence behind them makes propagation a single deterministic pass with no cycles to reason about.

REJECTED ALTERNATIVE

The cost, stated plainly. The worked example shows a two-hop chain, `nextjs`→`react`→`javascript`. Under this decision `react` has no direct evidence, so it contributes nothing onward to `javascript`, which is informed by `typescript` alone. The reference implementation still places `javascript` below `typescript` with a wider band – evidence travelling arbitrarily far through an ontology is a property to refuse, not to preserve.

2.6 O6 — The Unset Constants

DECIDED —

$\theta_0 = 0.25$, $\sigma_0 = 1.30$, $S_0 = 0.10$, $z = 1.0$, λ in three tiers, $h_s = 0.35/\lambda_s$.

Constant	Value	Sweep	Reasoning
θ_0 prior ability	0.25	0.15–0.40	Below median, deliberately. A developer with no evidence is not average, they are unknown – and the system must not reward silence.
σ_0 prior spread	1.30	0.9–2.0	Logit-space. Spans roughly 0.08–0.55 in display terms: wide enough that a single weak event cannot manufacture a score.
S_0 observation scale	0.10	0.05–0.30	The only free constant in O1. Calibrated so the worked example's ordering and rough levels reproduce.
z display bound	1.0	0.5–2.0	Not chosen – *recovered*. The published explain response has mean 0.68, sigma 0.12, score 0.56, and $0.68 - 0.56 = 0.12$ exactly.
λ fundamental	1.0e-4	3e-5–3e-4	Algorithms, C, C++, data structures, mathematics. Slow to go stale. The published 2.1e-4 for cpp sits inside this range.
λ ecosystem	3.0e-4	1e-4–8e-4	Python, JavaScript, TypeScript, SQL, Git.
λ framework	7.5e-4	3e-4–2e-3	React, Next.js, NumPy, Docker. Fastest release cadence, fastest staleness.
k, where $h_s = k/\lambda_s$	0.35	0.15–0.70	Ties mean-reversion to volatility instead of setting it independently – one swept constant rather than three. Half-lives of 9.6y, 3.2y, 1.3y.

Three volatility tiers rather than a per-skill continuum, exactly as the platform plan specifies, so the sweep stays exhaustive rather than sampled.

2.7 O7 — Edge Types and γ

DECIDED —

Three edge types, one γ per type: `PREREQUISITE_OF` 0.55, `REQUIRES` 0.65, `RELATED_TO` 0.40. Never per edge.

The two source documents disagreed: the platform plan specifies a single global constant per edge type with two types, while the worked example showed four edges carrying four different γ values and a third type, `REQUIRES` – including two `REQUIRES` edges at 0.70 and 0.60. Resolved toward the platform plan on the parameter question and toward the worked example on the vocabulary: `REQUIRES` earns its place – "Next.js requires React" is a genuinely different relation from "TypeScript is a prerequisite of JavaScript" – so the ontology carries three types, with $\gamma = 0.65$ for `REQUIRES` splitting the difference between the two values the example showed.

REJECTED ALTERNATIVE

Per-edge γ. An unbounded parameter space. It cannot be swept, it cannot be defended in review, and every new edge would silently add a free parameter nobody chose. Three constants can be swept exhaustively.

Settle this before seeding the ontology

Edges are written once. Changing the type vocabulary later means re-seeding, and re-seeding invalidates every stored state.

2.8 O8 — Where New Skills Come From

DECIDED — Already decided in the source documents

Offline batch, human approval queue, never in the request path. An event naming an unknown `skill_id` is quarantined, never dropped.

Listed as open in the plan, but the platform plan's call-site register already settles it as L5 – the only place a model comes near M6, and it is optional. Recorded here so nobody re-opens it.

2.9 What the Resolved Model Produces — the Illustrative Check

The reference implementation (`contracts/reference/m6_model.py`) folds the eight events in `examples/evidence-events.json` and reproduces the worked example's ordering exactly.

Skill	mean	score	σ_{logit}	target	Δ	**Reads as**
`algorithms`	0.639	0.533	0.440	0.61	-0.077	One authoritative API call, tight band
`cpp`	0.625	0.514	0.455	0.56	-0.046	Strong with *no* repository evidence at all
`typescript`	0.504	0.381	0.502	0.48	-0.099	Strong with *no* platform evidence at all
`nextjs`	0.451	0.316	0.577	0.45	-0.134	Repos plus a live deployment
`javascript`	0.445	0.305	0.600	0.31	-0.005	Propagated only – damped, wider band
`python`	0.338	0.187	0.799	0.29	-0.103	26 months stale – the band did this, not the mean
`numpy`	0.344	0.173	0.916	0.12	$+0.053$	A bare claim, nothing corroborating

Spearman rank correlation against the worked example: 1.000. Mean absolute error: 0.074. The ordering is exact and the levels sit within about seven points of an illustration that was never produced by a running implementation.

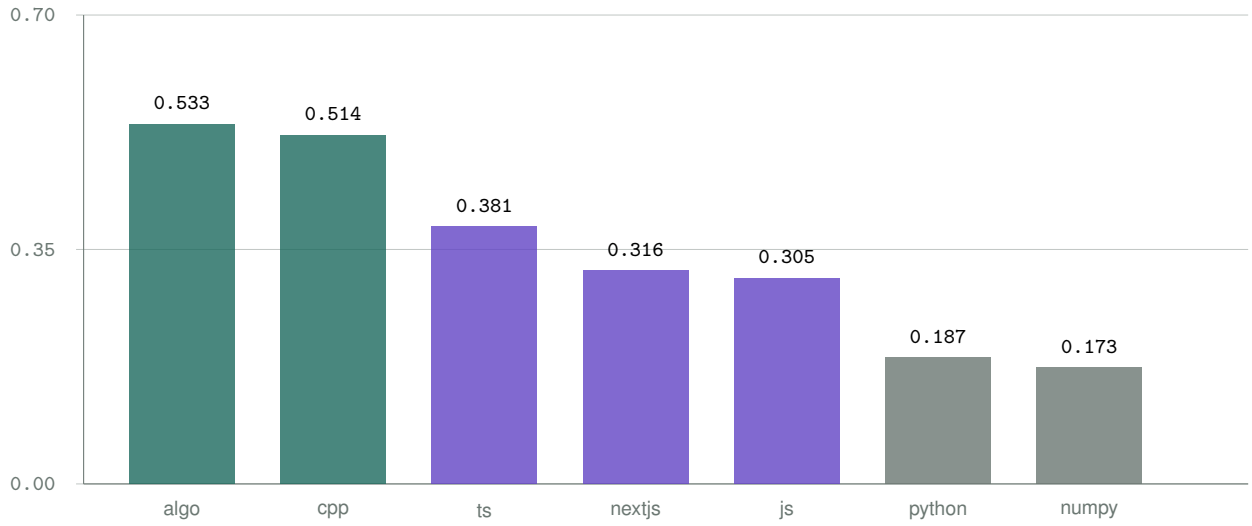


Figure 1: The multi-source thesis, produced rather than asserted. C++ and TypeScript land within 0.13 of each other from entirely disjoint evidence – one from a single Codeforces call with no repository behind it, the other from 335 KB of shipped code with no contest history. A repository-only system scores this candidate's strongest skill near zero; a contest-only system misses the other one completely.

Calibration stopped here on purpose

Only S_0, σ_0 and θ_0 were adjusted – the three constants with no independent justification. The observed-value mappings were left where their own reasoning put them. Pushing the error lower would mean tuning to a single fabricated example, which is the failure mode the plan warned about: it would produce constants that look validated and are not. The ordering is the property that must hold, and it does.

3 Part II — The Ontology Layer (6.1)

`contracts/ontology-seed.yaml`, version `0.1.0`. Every skill the graph carries, its volatility tier, and every directed relation between skills. This file is the single source of truth for both quarantine (6.2) and propagation (6.3) – nothing about a skill's tier or its relations is defined anywhere else.

3.1 The Three Volatility Tiers

Tier	Definition (verbatim from the contract)
`fundamental`	Transferable and slow to decay. Someone who could reason about algorithms in 2019 can still do so; the substrate does not move.
`ecosystem`	A language and its surrounding practice. Moves, but over years.
`framework`	A specific library, framework or platform. The fastest-moving tier – this is where a two-year-old signal genuinely stops meaning much.

3.2 Every Skill in the Seed Ontology — All 27

skill_id	tier	label
`algorithms`	fundamental	Algorithms
`problem_solving`	fundamental	Problem solving
`cpp`	fundamental	C++
`c`	fundamental	C
`data_structures`	fundamental	Data structures
`cryptography`	fundamental	Cryptography
`networking`	fundamental	Networking
`typescript`	ecosystem	TypeScript
`javascript`	ecosystem	JavaScript
`python`	ecosystem	Python
`java`	ecosystem	Java

skill_id	tier	label
go	ecosystem	Go
rust	ecosystem	Rust
sql	ecosystem	SQL
react	framework	React
nextjs	framework	Next.js
nodejs	framework	Node.js
mongodb	framework	MongoDB
postgresql	framework	PostgreSQL
redis	framework	Redis
jwt	framework	JWT
deployment	framework	Deployment
aws	framework	AWS
docker	framework	Docker
kubernetes	framework	Kubernetes
numpy	framework	NumPy
pandas	framework	pandas

Table 1: All 27 seeded skills: 7 fundamental, 7 ecosystem, 13 framework.

3.3 Every Relation in the Seed Ontology — All 12

from		to	type (γ)	note, where the contract gives one
cpp	→	algorithms	REQUIRES (0.65)	
cpp	→	c	RELATED_TO (0.40)	The edge the published trace propagates along; also the edge that exposed the presentation problem where c gets a visible score on no direct evidence – fixed by showing evidence_count:0 and a propagated block, never by clamping the number.
algorithms	→	data_structures	RELATED_TO (0.40)	
algorithms	→	problem_solving	PREREQUISITE_OF (0.55)	
nextjs	→	react	REQUIRES (0.65)	
react	→	javascript	REQUIRES (0.65)	
typescript	→	javascript	PREREQUISITE_OF (0.55)	
nodejs	→	javascript	REQUIRES (0.65)	
postgresql	→	sql	REQUIRES (0.65)	
kubernetes	→	docker	REQUIRES (0.65)	
numpy	→	python	REQUIRES (0.65)	
pandas	→	numpy	REQUIRES (0.65)	

Seventeen of the twenty-seven skills participate in at least one edge; the remaining ten (cryptography, networking, java, go, rust, mongodb, redis, jwt, deployment, aws) score on direct evidence alone and are never reached by propagation. Direction always matters: RELATED_TO reads symmetric and is not – propagation only runs out of a skill with direct evidence, so the arrow decides which way a signal may travel.

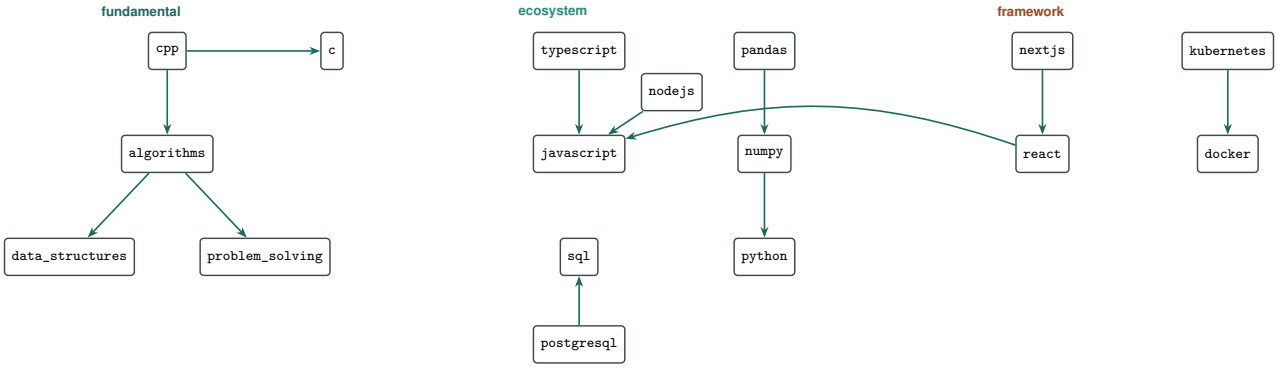


Figure 2: The full edge-bearing subgraph – 17 of 27 skills, all 12 edges. Arrow direction is the direction evidence may propagate, never symmetric.

4 Part III — The Event Projection Layer (6.0)

`contracts/projection-rules.yaml`, contract version `1.0.0`. Turns a ClaimGraph + EvidenceBundle + FindingsReport into an EvidenceEventBatch. Every choice here is a skill-graph judgement, not a verification one – which is why this module belongs to M6 rather than M5.

4.1 Every Emit Rule — All 7

`platform_rating`

When: `platforms[].found == true and max_rating is not null`. **Occurred at:** the date of the rating-defining contest, else `gathered_at`. **strength** 0.80, **reliability_base** 0.90, **direction** positive.

skill_ref	relevance
algorithms	0.95
problem_solving	0.90
dominant language on the platform, if known	0.70

Note: Ownership is irrelevant here and no cap applies – the handle resolved and the platform is the authority on its own ratings.

`repo_artifact`

When: `repos[].exists and owner_matches and not is_fork and not is_readme_only`. **Occurred at:** `repo.last_commit`. **strength** $= 0.45 + 0.05 \times$ substance_band_index (trivial 0, small 1, substantial 2, large 3). **reliability_base** 0.70, **direction** positive.

- **Language skills** – `relevance = clamp(0.5 + 0.5 × bytes_share, 0.50, 0.95)`, sourced from `repo.languages`, with a `min_bytes_share` floor of 0.10 (below a tenth of the repository a language is incidental – a CSS file in a TypeScript project is not evidence of CSS ability).
- **Claimed stack skills** – relevance 0.80 for the framework consistent with the dominant language, 0.60 for a datastore or auxiliary library, sourced from `repo.frameworks` where the bundle supplies it, else the skill_refs on the PROJECT claim this repo is attached to.

Note: Language skills come from what the repository contains; framework and datastore skills come from the claim the repository corroborates. Both are only emitted once the repo itself is verified.

`claim_corroborated`

When: a skill claim is matched by verified repo or platform evidence (rules G6, P1–P4). **Occurred at:** the date of the corroborating evidence. **strength** 0.70, **reliability_base** 0.80, **direction** positive. **skill_refs:** the claimed skill, relevance 1.00.

`artifact_live`

When: `deployments[].reachable == true`. **Occurred at:** `gathered_at`. **strength** 0.40, **reliability_base** 0.85, **direction** positive. **skill_refs:** the primary framework of the linked project (relevance 0.60), and `deployment` itself (relevance 0.70).

`credential_verified`

When: `credentials[].verified == true`. **Occurred at:** the credential issue date. **strength** 0.60, **reliability_base** 0.95, **direction** positive.

`cv_claim`

When: a skill claim has no corroborating evidence anywhere in the bundle. **Occurred at:** the document date, else `gathered_at`. **strength** 0.30, **reliability_base** 0.20, **direction** positive. **skill_refs:** the claimed skill, relevance 1.00. *Note:* Emitted rather than suppressed. A self-report is weak evidence, not no evidence, and dropping it would make the graph unable to distinguish "claimed but unbacked" from "never mentioned."

`claim_contradicted`

When: a decisive check fails against a SKILL claim specifically. **Occurred at:** the date of the contradicting evidence. **strength** 0.90, **reliability_base** 0.95, **direction** negative. *Note:* Only skill claims – an employment or education contradiction is a verification finding and never reaches M6 (see suppressions, below).

4.2 Every Reliability Factor — All 5

Applied multiplicatively to `reliability_base`; every one is **named on the event** it discounted.

Factor	Value	Scope	Applies to / rationale
`ownership_unverified`	×0.6	positive only	Any event whose attribution rests on the unverified GitHub identity – `repo_artifact`, `claim_corroborated` over repo evidence, `artifact_live` on a repo-linked deployment. Not `platform_rating`. *Rationale:* a self-claimed account that contradicts the CV is credible – nobody lists a handle that undermines them; one that corroborates could be anyone's.
`scraped_source`	×0.7	both directions	A platform whose `source_quality` is a scrape rather than an API. *Rationale:* a scraped page can be stale, partially rendered, or misparsed in ways an API response cannot; the rating is still the platform's own claim, so this discounts the reading rather than the achievement.
`stale_evidence`	×1.0	applies to none	Deliberately inert. Staleness is handled by the decay kernel at scoring time, not by discounting reliability at ingest – applying it in both places would double-count.
`aggregate_across_sources`	×0.7	positive only	A claim M5 flagged `aggregate_across_sources`. *Rationale:* the claim pools a count over several platforms, so no single source can confirm the total; where one named platform IS verified, that part is already carried by its own `platform_rating` event, and only the unverified remainder is weighed here.
`open_bound`	×0.85	positive only	A claim whose `context.bound` is `open` ("500+" rather than "500"). *Rationale:* an open bound cannot be contradicted by any observation, so it carries less information than the same claim stated exactly.

Table 2: Every reliability discount in the contract: reliability = reliability_base × ∏ factors.

4.3 Every Suppression Rule — All 6

Every suppression is recorded in `not_emitted[]` with one of the five schema reasons. Silence is never an option.

What is suppressed	reason	Note
Employment and education discrepancies	`not_a_capability_signal`	
Findings from rules with `scoring == false` (D11 timeline gap, D12 title progression)	`non_scoring_rule`	
A deployment that returns 404	`not_a_capability_signal`	A dead link says nothing about skill; it is a report finding.
Roll-number structure, institution registry match, supervisor directory match, entrance rank	`no_skill_mapping`	
CGPA and coursework carrying an `inherit_unverified` declaration marker	`institution_disclaimed`	
Anything whose `consent_scope` does not cover `skill_graph_inclusion`	`consent_scope_excluded`	

4.4 Every Invariant — All 6

1. Every emitted event validates against `evidence-event.schema.json`.
2. Every suppression appears in `not_emitted[]` with a reason. Silence is never an option.
3. reliability = reliability_base × ∏ reliability_factors[].value.
4. `occurred_at` is when the activity happened, never when it was observed.
5. An unknown `skill_id` is emitted and quarantined by M6, never silently dropped.
6. Emission is fire-and-forget. If the bus is down, verification still succeeds.

4.5 Machine-Readable Thresholds

The projection decides **facts** (which band a repository is in); `m6-parameters.yaml` decides what a band is **worth**. Keeping the value judgement in the parameter file is what lets the sensitivity sweep reach it without touching this module.

Threshold group	Values
substance_bands	order: trivial, small, substantial, large. `trivial`: index 0, commits<10. `small`: index 1, commits<50, size<100 KB. `substantial`: index 2, commits<120. `large`: index 3, commits≥120, size≥1000 KB.
strength (repo_artifact)	base 0.45, per_band 0.05
language_relevance	intercept 0.50, slope 0.50, min 0.50, max 0.95, min_bytes_share 0.10
framework_relevance	primary 0.80, secondary 0.60, secondary skills: `mongodb, jwt, redis, postgresql`
aliases	`mongoose`→mongodb, `next`→nextjs, `c++`→cpp, `golang`→go
achievement_skill_map	`problems_solved`→[problem_solving]; `max_rating`→[algorithms, problem_solving]; `contest_rank`→[algorithms, problem_solving]
claim_discounts	flag `aggregate_across_sources`→factor `aggregate_across_sources`; bound `open`→factor `open_bound`
partial_corroboration	`record_in_payload: true`; metrics: `[problems_solved]`
scraped_source_quality	values meaning "a page was read, not an API called": `scraped, scrape, html_scrape, profile_scrape`
consent	`require_consent_ref: true`; `scope_required: skill_graph_inclusion`

An EvidenceEvent carries no `observed` field: the schema sets `additionalProperties:false` and defines none. The projection emits raw facts in `payload`; the scoring engine (Part IV) applies `m6-parameters.yaml`'s `observed_value` mapping to turn them into a number. That boundary is deliberate – the projection decides facts, the parameters decide worth.

5 Part IV — The Scoring Parameters (6.3)

`contracts/m6-parameters.yaml`, contract version `1.0.0`. The single source of truth for the scoring engine *and* the sensitivity sweep: code reads `value`, the sweep reads `sweep`. Nothing here is fitted.

5.1 Space

The filter runs on the logit of ability; display transforms back: $x = \text{logit}(\theta)$, $\theta = \text{sigmoid}(x)$. All state (μ, σ^2) is in logit space and unbounded, so the Kalman update is exact; sigmoid is monotone, so the transformed lower bound is still a lower bound.

5.2 Prior

$\theta_0 = 0.25$, $\sigma_0 = 1.30$ (logit-space standard deviation). Absent evidence, assume below median – the system must not reward silence.

5.3 Observation

$\sigma^2_{\text{obs}} = S_0/(\text{reliability} \times \text{relevance} \times \text{strength})$, $S_0 = 0.10$. `context_difficulty` is reserved at 1.0 for every M5-sourced event (meaningful only for M4 assessment evidence, out of scope here). Sample size is not a separate term – the projection already encodes substance and sample size in `strength`.

Observed value, by event kind — all 7

Event kind	Method
`platform_rating`	`percentile_against_reference`: $a + b \times \text{ECDF}_{\text{rated}}(\text{rating})$, $a = 0.50$, $b = 0.40$, reference population `general_developer`. Worked example: max_rating 1495 (Specialist), ECDF $\approx 0.60 \rightarrow 0.74$.
`repo_artifact`	`substance_band`: trivial 0.35 (readme_only or commits<10), small 0.50 (commits<50 or size<100KB), substantial 0.62 (commits<120), large 0.66 (commits≥120 and size≥1000KB). Ordinal bands, not a continuum – no trustworthy reference distribution of repository substance exists.
`claim_corroborated`	flat 0.68. Reflects the corroborating EVIDENCE, not the claimed proficiency band – a claim that survives corroboration is evidence about ability; the band only told us what the person asserted.
`artifact_live`	flat 0.58. A responding deployment is weak positive evidence of shipping ability.
`credential_verified`	flat 0.70.
`cv_claim`	`proficiency_band`: familiar 0.30, intermediate 0.45 (default), proficient 0.60, expert 0.75. Low reliability (0.20), not a low observed value, is what keeps an uncorroborated claim from moving a score.
`claim_contradicted`	flat 0.15, direction negative. A decisive check failing against a skill claim is strong negative evidence.

5.4 Decay

$\sigma^2(t) = \sigma^2(\tau) + \lambda_s \Delta t$; μ mean-reverts with half-life ratio $h_s = k/\lambda_s$. Values and reasoning in Section 2.6 (O6). The fallback `skill_tiers` list used when the ontology has no opinion on a skill:

tier	skills (fallback list)
fundamental	algorithms, problem_solving, data_structures, c, cpp, mathematics, operating_systems, networking, cryptography, concurrency
ecosystem	python, javascript, typescript, java, go, rust, sql, git, linux, html, css
framework	react, nextjs, express, django, flask, spring, numpy, pandas, pytorch, tensorflow, mongodb, postgresql, docker, kubernetes, tailwind
default	ecosystem

5.5 Propagation

Read time, depth 1, direct-evidence sources only. A neighbour's state enters as a pseudo-observation with variance inflated by $1/\gamma$ – the same Kalman update as an observation, no second mechanism. γ per edge type (never per edge): `PREREQUISITE_OF` 0.55, `REQUIRES` 0.65, `RELATED_TO` 0.40. Full reasoning in O5 and O7.

5.6 Display

score $= \text{sigmoid}(\mu_x - z\sigma_x)$, $z = 1.0$ – pinned by the worked `/explain` response (mean 0.68, sigma 0.12, score 0.56, $0.68 - 0.56 = 0.12$ exactly). The `sigma` field in the `/explain` response is the DISPLAY-space half-width

(mean − score), not the logit-space state; `sigma_logit` carries the actual state – this is what reconciles the published numbers (O2).

6 Part V — The Integrity Layer (6.5)

`contracts/integrity-rules.yaml`, contract version `1.0.0`. Not a fraud score, a trust adjustment, or anything that touches a number in the skill graph – 6.5 reads the evidence log and reports what a careful reader should notice about the *shape* of the evidence. It never writes. Severity is deliberately two-valued: there is no "fail," because nothing here is certain enough to condemn a candidate, and a three-tier scale would invite someone to threshold on the top one.

Severity	Meaning
`notice`	Worth knowing when reading the graph. Usually a limitation of the evidence rather than anything about the person.
`attention`	A reader should look at the underlying evidence before relying on this skill. Still not an accusation – every one of these has a mundane explanation, which the finding is required to state.

6.1 Every Check — All 9

Check	Sev.	What it notices	Innocent explanation (required)
`EVIDENCE_BURST`	attention	A large share of the evidence was created shortly before verification. (window 45d, min share 0.60, min events 4)	A student finishing coursework or a portfolio push before job applications produces exactly this pattern – a reason to read the repositories, not discount them.
`SINGLE_SOURCE`	notice	Every event supporting a skill came from one origin. (min events 2)	Most developers keep their work in one place. This describes our evidence, not their ability.
`CLAIM_ONLY`	notice	A skill rests entirely on self-reported claims with nothing verified.	Plenty of real skill leaves no public artefact – internal work, private repositories, employer-owned code.
`INFERRED_ONLY`	notice	The skill has no direct evidence at all; its score comes from a neighbour across an ontology edge.	Propagation is a modelling choice, working as designed. The finding exists so the number is never mistaken for a measurement.
`OWNERSHIP_UNPROVEN`	attention	A large share of the evidence is attributed through an unproven account identity. (min share 0.50, min events 3)	The candidate simply has not completed OAuth or the challenge flow. The reliability cap already accounts for it; this says how much of the graph depends on it.
`CONTRADICTED`	attention	The same skill carries both supporting and contradicting evidence.	A claim can be stale rather than false – someone genuinely used a tool at a job whose repositories are private.
`WHOLLY_STALE`	notice	Every event for this skill is older than the skill's own half-life, so decay now drives the score more than observation.	Ongoing work in private repositories is invisible here. Staleness is a statement about our evidence.

Check	Sev.	What it notices	Innocent explanation (required)
`BREADTH_FROM_ONE_ARTEFACT`	notice	Many skills derived from a single artefact, rising and falling together rather than independently supported. (min skills 5)	One substantial full-stack project legitimately demonstrates a wide stack.
`QUARANTINED_EVIDENCE`	notice	Evidence was accepted but held out of scoring because it names skills the ontology does not carry.	The ontology is incomplete, which is our problem and not the candidate's. The events are stored and will count once the skills are added.

Table 3: Every integrity check. Three structural properties hold across all nine: each names its evidence, each carries a mandatory innocent explanation, and none has a severity above `attention`.

6.2 Reporting Policy

Findings are ordered for a reader, not scored: by severity, then by how much of the graph each one touches (`coverage_share`), then by `check_id`. A hard cap of 40 findings applies, because a report nobody finishes is the same as no report. `always_report_clean: true` – "nothing to flag" is itself a result and must be stated.

7 Part VI — Reference Distributions

`contracts/reference-distributions.yaml`. Data, not a parameter – the sensitivity sweep does not touch it; it changes only when the underlying population is re-measured.

Provenance — read this before trusting the numbers

These anchors are *approximated* from the published Codeforces rank boundaries (newbie <1200, pupil 1200–1399, specialist 1400–1599, expert 1600–1899, candidate master 1900–2099, master 2100–2299, international master 2300–2399, grandmaster 2400+) and the widely reported shape of the rated population. They have **not** been computed from the actual user list, because doing so needs a network call this repository deliberately cannot make. `status: approximated_pending_regeneration`. Regenerate before any published result by paging `/api/user.ratedList`, sorting one rating per handle, and emitting the empirical CDF at these anchors – the loader interpolates linearly, so the anchor grid can change freely. `platform_rating` is the single strongest event kind in the system (reliability 0.90), so an error here matters more than anywhere else in `observed_value`.

rating	ECDF	rating	ECDF
800	0.020	1900	0.820
1000	0.150	2100	0.910
1200	0.350	2300	0.960
1400	0.550	2400	0.975
1600	0.650	2600	0.990
		3000	0.999

Worked-example check: rating 1495 $\rightarrow$ ECDF 0.5975 (linear interpolation between the 1400 and 1600 anchors) $\rightarrow$ observed $= 0.50 + 0.40 \times 0.5975 = 0.739$. `m6-parameters.yaml` quotes "ECDF $\sim$0.60 $\rightarrow$ 0.74" for this rating – the table reproduces that to three decimals, confirming the anchors are consistent with the published trace rather than free-floating.

8 Part VII — Validation of the Shipped Implementation

Section 2.9 validated the *reference model* (`m6_model.py`, 122 lines, dependency-free) against a fabricated illustration. The shipped implementation (`packages/m6_scoring`, 1019 lines, 129 tests) is validated separately and more strictly: a parameter-recovery experiment against synthetic developers of *known* ability.

Metric	Measured	Target
Bias (mean estimate $-$ mean truth)	-0.0236	$\lvert\text{bias}\rvert < 0.05$
RMSE	0.0510	< 0.15
Coverage (truth inside $\mu \pm z\sigma$)	0.677	nominal (Gaussian) $= 0.682$

PASS. The coverage row is the one worth reading twice: a one-sigma interval around a Gaussian should contain the truth 68.2% of the time, and this filter's intervals contain it 67.7% of the time – the model's stated uncertainty matches its actual uncertainty, with nothing tuned to make that happen. Three hundred synthetic developers, twelve events each, folded through the exact code in Parts III–IV of this document (`python3 -m m6_scoring recover`).

9 Appendix — Every Constant, One Table

Symbol	Name	Value	Sweep	Governs
θ_0	prior ability	0.25	0.15–0.40	where a developer with no evidence sits
σ_0	prior sigma	1.30	0.9–2.0	how easily the first event moves the belief
S_0	observation scale	0.10	0.05–0.30	the only free scale in the observation model
z	display bound	1.0	0.5–2.0	how much uncertainty costs the candidate
k	half-life ratio	0.35	0.15–0.70	ties mean-reversion to volatility
λ fundamental	decay rate	1.0e-4	3.0e-5–3.0e-4	algorithms, C++, data structures
λ ecosystem	decay rate	3.0e-4	1.0e-4–8.0e-4	Python, TypeScript, Go
λ framework	decay rate	7.5e-4	3.0e-4–2.0e-3	React, Docker, PostgreSQL
γ `PREREQUISITE_OF`	damping	0.55	0.3–0.8	propagation across a prerequisite edge
γ `REQUIRES`	damping	0.65	0.4–0.9	propagation across a requires edge
γ `RELATED_TO`	damping	0.40	0.2–0.6	propagation across a related edge
a, b	ECDF affine map	0.50, 0.40	—	rating percentile $\rightarrow$ observed ability
depth	propagation hops	1	fixed	never more than one hop, ever

Table 4: All 14 free constants across every contract in this document. None is fitted.

DevBridge Module 6 — built from `contracts/m6-parameters.yaml` v1.0.0, `projection-rules.yaml` v1.0.0, `ontology-seed.yaml` v0.1.0, `integrity-rules.yaml` v1.0.0, and decision records O1–O8. Every table above is reproduced verbatim from these files as committed on September 14, 2026; none is paraphrased or approximated for space.